MULTI-PLATFORM INVENTORY MANAGEMENT SYSTEM

with a RESTful API Backend

PROJECT DOCUMENTATION SET

Work Breakdown Structure · Software Requirements Specification
Project Chapter Report · Prototype Description
Gantt Chart / Schedule · User Manual

Build window: February – August 2026
Submitted: August 2026

Table of Contents

1. Work Breakdown Structure (WBS)

Project: Multi-Platform Inventory Management System with RESTful API Backend

Build window: February – August 2026 (implementation: June – August 2026)

1.1 WBS Hierarchy

```
1.0  Multi-Platform Inventory Management System
├── 1.1  Project Initiation & Planning
│   ├── 1.1.1  Problem definition and feasibility study
│   ├── 1.1.2  Stakeholder identification (Business Owner, Administrators,
│   │          Branch Managers, Storekeepers, Cashiers)
│   ├── 1.1.3  Scope statement and deliverable list
│   └── 1.1.4  Project schedule and risk register
│
├── 1.2  Requirements Engineering
│   ├── 1.2.1  Elicitation — interviews with shop staff, observation of manual processes
│   ├── 1.2.2  Functional requirements per module (SRS §3)
│   ├── 1.2.3  Non-functional requirements — security, performance, offline use (SRS §4)
│   └── 1.2.4  SRS review and sign-off
│
├── 1.3  System Design
│   ├── 1.3.1  Architecture design — REST API + web client + desktop client
│   ├── 1.3.2  Database design — PostgreSQL schema (60 tables)
│   ├── 1.3.3  API contract design — resource modelling, JWT auth flow
│   ├── 1.3.4  UI/UX wireframes — 32 application screens
│   └── 1.3.5  Offline synchronisation strategy (branch-scoped pull/push)
│
├── 1.4  Backend Implementation (RESTful API)
│   ├── 1.4.1  Project scaffolding — Express 5, layered architecture
│   │          (routes → controllers → services → repositories)
│   ├── 1.4.2  Security middleware — helmet, CORS, rate limiting, HPP,
│   │          XSS-clean, bcrypt hashing, express-validator
│   ├── 1.4.3  Authentication & authorisation
│   │   ├── 1.4.3.1  Login with access + refresh tokens
│   │   ├── 1.4.3.2  Branch selection with branch-scoped token re-issue
│   │   ├── 1.4.3.3  Role-based permissions (roles / role_permissions / permissions)
│   │   └── 1.4.3.4  Password reset with 1-hour expiring email tokens
│   ├── 1.4.4  Core inventory modules
│   │   ├── 1.4.4.1  Products, categories, suppliers, units of measure, images
│   │   ├── 1.4.4.2  Per-branch inventory (product_branch_inventory) with reorder levels
│   │   ├── 1.4.4.3  Stock movements / inventory transactions
│   │   └── 1.4.4.4  Inter-branch stock transfers
│   ├── 1.4.5  Purchasing module — purchase orders, items locked to DRAFT,
│   │          automatic total recalculation, receiving, supplier payments
│   ├── 1.4.6  Sales module — POS carts, sales, sale items, receipts, customer payments
│   ├── 1.4.7  Customers & expenses modules
│   ├── 1.4.8  Reporting module — daily/monthly/annual sales, profit,
│   │          inventory valuation, low stock; CSV / Excel / PDF export
│   ├── 1.4.9  Notification service — in-app, e-mail (Gmail SMTP over implicit
│   │          TLS), SMS fan-out; targeting USERS / ALL / BRANCH
│   ├── 1.4.10 Background message queue — persistent jobs, exponential-backoff
│   │          retries, admin monitoring + resend endpoint
│   ├── 1.4.11 Audit trail — audit logs, activity logs, login history
│   └── 1.4.12 Offline sync endpoints — branch-scoped incremental pull/push
│
├── 1.5  Web Client Implementation (React 19 + Vite + TypeScript)
│   ├── 1.5.1  Auth context, refresh-token handling, route guards
│   ├── 1.5.2  Branch selection screen and branch-aware data loading
│   ├── 1.5.3  Dashboard with KPI cards and charts
│   ├── 1.5.4  POS terminal — barcode search, live branch-stock display, checkout
│   ├── 1.5.5  Inventory screens — products, stock movements, transfers, purchases
│   ├── 1.5.6  Administration screens — users, roles, branches, business settings
│   ├── 1.5.7  Notifications centre + compose page (USERS/ALL/BRANCH targets)
```

```
    ├── 1.5.8  Reports screen with export actions
    └── 1.5.9  Admin message-queue monitor with resend action

├── 1.6  Desktop Client Implementation (Electron 42)
    ├── 1.6.1  Electron shell, build pipeline (electron-builder), HID scanner support
    ├── 1.6.2  IndexedDB offline cache (idb) — branch-scoped entities only
    ├── 1.6.3  Offline POS — cached catalog, queued mutations, auto re-sync
    └── 1.6.4  Sync log viewer for conflict/outage diagnosis

├── 1.7  Testing & Quality Assurance
    ├── 1.7.1  Unit/integration tests (Jest + Supertest)
    ├── 1.7.2  Endpoint verification against live database (per-feature scripts)
    ├── 1.7.3  Front-end linting (oxlint) and type-checking (tsc -b)
    ├── 1.7.4  Role-based permission testing
    └── 1.7.5  Defect fixing — e.g., snake/camelCase filter normalisation,
               SMTP transport fix (587 STARTTLS → 465 implicit TLS),
               queue retry-status bug, PO total recalculation backfill

├── 1.8  Deployment & Release
    ├── 1.8.1  Environment configuration (.env management)
    ├── 1.8.2  Database migration/backfill scripts
    └── 1.8.3  Desktop installer packaging

└── 1.9  Documentation & Handover
    ├── 1.9.1  Work breakdown structure (this document)
    ├── 1.9.2  Software Requirements Specification
    ├── 1.9.3  Project chapter report
    ├── 1.9.4  Prototype description
    ├── 1.9.5  Gantt chart / schedule
    └── 1.9.6  User manual
```

1.2 Task Table

WBS	Task	Primary output	Est. effort
1.1	Initiation & planning	Scope statement, schedule	1 week
1.2	Requirements engineering	SRS v1.0	2 weeks
1.3	System design	ERD, architecture diagrams, wireframes	3 weeks
1.4	Backend implementation	RESTful API (30+ route modules)	8 weeks
1.5	Web client	32-screen React application	6 weeks
1.6	Desktop client	Offline-capable Electron app	3 weeks
1.7	Testing & QA	Test scripts, defect log, fixed build	3 weeks
1.8	Deployment	Configured environments, installer	1 week
1.9	Documentation	This documentation set	2 weeks

Effort overlaps between tracks; totals exceed calendar duration because backend, web and desktop work ran concurrently from June to August 2026.

1.3 Milestones

#	Milestone	Target month	Verification
M1	Requirements approved (SRS)	March 2026	Sign-off
M2	Database + API skeleton running	April 2026	Server boots, migrations applied
M3	Feature-complete backend	July 2026	All endpoints verified against live DB
M4	Web client feature-complete	July 2026	Full workflow walkthrough
M5	Offline desktop client working	August 2026	Airplane-mode POS test
M6	QA passed / defects closed	August 2026	Lint + type-check clean, regression suite green
M7	Documentation set delivered	August 2026	docs/ complete

1.4 Roles & Responsibilities

Role	Responsibility
Project lead / developer	Architecture, backend, clients, documentation
Business Owner (client)	Domain rules approval, UAT participation
End users (managers, storekeepers, cashiers)	Requirements input, acceptance testing

1.5 Risks & Mitigations (observed during the project)

Risk	Impact	Mitigation taken
Unreliable shop internet	POS downtime	Offline-first desktop cache + sync queue
SMTP blocking on some networks	E-mail delivery failure	Implicit-TLS port 465 + logged fallback + queue retries
Data entry of inactive records	Confusing catalogs	Soft-delete (is_active) + hidden-by-default listings
Concurrent stock edits across branches	Overselling	Per-branch inventory rows + transactional updates + availability checks

2. Software Requirements Specification (SRS)

Standard: IEEE 830 (adapted)

Version: 1.0 — August 2026

2.1 Introduction

2.1.1 Purpose

This document specifies the functional and non-functional requirements of the Multi-Platform Inventory Management System (MPIMS): a RESTful-API-driven inventory platform with a browser client and an offline-capable desktop client used by retail businesses operating multiple branches.

2.1.2 Scope

The system provides:

- Multi-branch inventory tracking with per-branch stock, reorder levels and full stock-movement history.
- Point-of-Sale (POS) checkout with barcode support, including offline operation on the desktop client.
- Purchasing (purchase orders → receiving → supplier payments).
- Inter-branch stock transfers.
- Customer management and customer payments.
- Role-based administration of users, branches, roles and permissions.
- Notifications via in-app, e-mail and SMS channels with audience targeting.
- Business reporting with CSV/Excel/PDF exports.
- A background message queue for reliable asynchronous message delivery.

Out of scope: accounting/general ledger, payroll, e-commerce storefronts.

2.1.3 Definitions & Acronyms

Term	Meaning
MPIMS	Multi-Platform Inventory Management System
PO	Purchase Order
QoH	Quantity On Hand
PBI	product_branch_inventory — per-branch stock row
RBAC	Role-Based Access Control

2.1.4 References

IEEE 830-1998; project WBS (01-work-breakdown.md); source code in this repository.

2.2 Overall Description

2.2.1 Product Perspective

A three-tier system: a Node.js/Express RESTful API over PostgreSQL, consumed by a React web client (browser) and an Electron desktop client that can operate offline against a local IndexedDB cache and re-synchronise later.

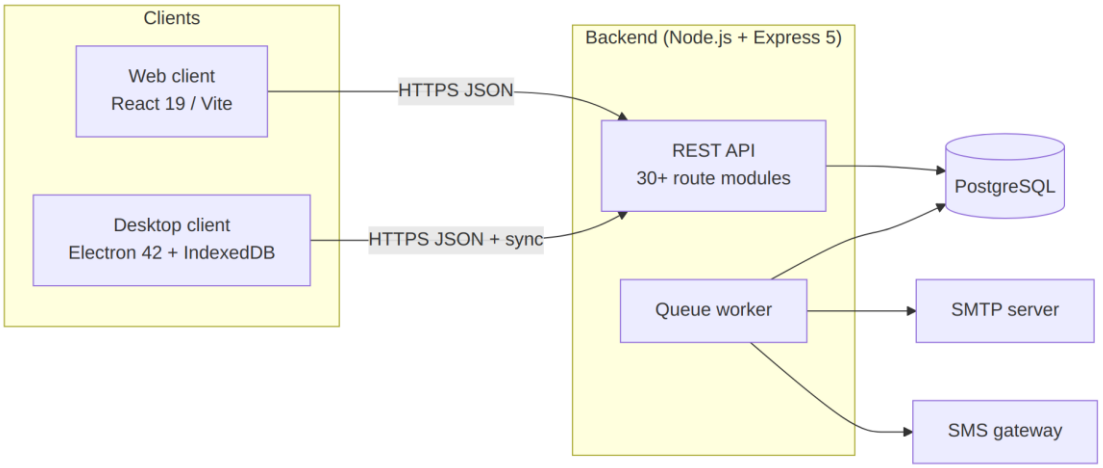

Figure 2.1 — System context: web and desktop clients against the REST API

2.2.2 Product Functions (summary)

Authentication & branch selection; product catalogue management; multi-branch inventory; stock movements; transfers; purchasing; POS sales; customers; expenses; notifications; reporting; user/role administration; audit logging; offline synchronisation; background job processing.

2.2.3 User Classes & Characteristics

User class	Typical duties	Key screens
Business Owner	Oversight across all branches, settings	Dashboard, Reports, Branches, Users
Administrator	System configuration, accounts, queue monitoring	Users, Roles, Message Queue, Audit Logs
Branch Manager	Branch operations oversight	Inventory, Purchases, Transfers, Low Stock
Storekeeper	Receiving goods, stock counts/movements	Purchases, Stock Movements, Products
Cashier	Selling to walk-in customers	POS, Sales

2.2.4 Operating Environment

- Server: Node.js ≥ 20, PostgreSQL ≥ 14, Windows/Linux/macOS.

- Clients: modern browsers (Chrome/Edge/Firefox); Windows desktop via Electron installer; optional HID barcode scanners (node-hid).
- Network: HTTPS required for clients; system remains usable offline on the desktop client for cached branch data.

2.2.5 Design & Implementation Constraints

- REST/JSON over HTTP(S) only; stateless access tokens (JWT) with refresh rotation; tokens are branch-scoped after selection.
- All monetary values recorded per sale/payment rows; totals recalculated server-side.
- Deletion is soft (is_active = false) — historical references must survive.
- E-mail delivery must degrade gracefully (queue + log fallback), never block the triggering request.

2.2.6 Assumptions & Dependencies

- Each business has at least one active branch; users may belong to many branches.
- Gmail SMTP credentials (app password) available for outbound mail.
- Desktop devices have local storage sufficient for one branch's catalog cache.

2.3 Specific Requirements (Functional)

Requirement IDs use FR-<module>-<n>. "Shall" denotes mandatory behaviour.

2.3.1 Authentication & Session Management (AUTH)

- FR-AUTH-1 The system shall authenticate users with username/e-mail + password (bcrypt-hashed) and issue an access token and refresh token.
- FR-AUTH-2 After login, the user shall select an active working branch; the API shall re-issue tokens embedding the selected branch context.
- FR-AUTH-3 If a user has exactly one assigned branch, it shall be selected automatically.
- FR-AUTH-4 Refresh tokens shall rotate on use and be revocable per session.
- FR-AUTH-5 The system shall support self-service password reset using a single-use token delivered by e-mail, valid for one hour.
- FR-AUTH-6 Login history (IP, device) shall be recorded.

2.3.2 Authorisation (RBAC) (PERM)

- FR-PERM-1 Permissions shall be grouped into named roles and assignable per user.
- FR-PERM-2 Protected endpoints shall enforce permission checks (e.g., MANAGE_NOTIFICATIONS) in addition to authentication.
- FR-PERM-3 Administrators shall manage roles and their permission sets through the UI.

2.3.3 Catalogue Management (CAT)

- FR-CAT-1 CRUD for products (SKU, barcode, name, base UoM, supplier, category), categories, suppliers and units of measure.
- FR-CAT-2 Product images shall be uploadable; one primary image shown in lists.
- FR-CAT-3 Deactivated (soft-deleted) records shall be hidden from default listings but restorable; listings may opt in to include inactive items.
- FR-CAT-4 Price changes shall be tracked in a price history table.

2.3.4 Multi-Branch Inventory (INV)

- FR-INV-1 Stock levels shall be maintained per product per branch (QoH, reorder level, reorder quantity).
- FR-INV-2 Listing endpoints scoped to the signed-in branch by default; explicit branch filters accepted in both camelCase and snake_case keys.
- FR-INV-3 Stock movements (IN/OUT/ADJUSTMENT/TRANSFER) shall append-only record quantity, reference and actor, and update PBI atomically.
- FR-INV-4 Stock-out requests shall validate availability and reject quantities exceeding available stock.

- FR-INV-5 Transfers shall move stock between two branches, decrementing source and incrementing destination within one transaction.
- FR-INV-6 The system shall flag low-stock items (QoH ≤ reorder level).

2.3.5 Purchasing (PO)

- FR-PO-1 Create purchase orders per branch with line items (product, quantity, unit cost).
- FR-PO-2 Items shall be addable/editable/removable only while the PO is in DRAFT status; attempts otherwise shall fail with HTTP 409.
- FR-PO-3 PO totals shall be recalculated automatically after every item change (no stale totals).
- FR-PO-4 Receiving a PO shall increase destination-branch stock and log inventory transactions.
- FR-PO-5 Supplier payments shall be recordable against POs/suppliers.

2.3.6 Sales / POS (POS)

- FR-POS-1 The POS shall search products by name/SKU/barcode within the current branch and show live availability.
- FR-POS-2 Checkout shall create a sale with items, decrement branch stock transactionally and produce a receipt record.
- FR-POS-3 Barcode input shall be supported (manual entry, scanner via keyboard wedge or HID on desktop).
- FR-POS-4 Offline (desktop): sales shall be creatable against the local cache and queued for synchronisation; conflicts reported in Sync Logs.
- FR-POS-5 Customer payments and outstanding balances shall be trackable per customer.

2.3.7 Notifications (NOTIF)

- FR-NOTIF-1 Authenticated staff shall compose notifications with title, message, type and channels (in-app, e-mail, SMS).
- FR-NOTIF-2 Recipients shall be selectable as specific users, ALL active users, or all active users of a chosen branch (target: USERS/ALL/BRANCH), resolved at send time.
- FR-NOTIF-3 Delivery shall occur through the persistent message queue with retry/backoff; each recipient gets independent read-state.
- FR-NOTIF-4 Users shall view their notifications, mark them read and delete them.

2.3.8 Background Message Queue (QUEUE)

- FR-QUEUE-1 Every e-mail/SMS send shall be enqueued persistently before delivery is attempted.
- FR-QUEUE-2 Failed jobs shall retry with exponential backoff (attempt² × 30 s, capped at 1 h) up to a configurable maximum; exhausted jobs become FAILED with the last error stored.
- FR-QUEUE-3 A retried (requeued) job shall return to PENDING so the worker can claim it again.

- FR-QUEUE-4 Administrators shall list jobs filtered by status/type and resend PROCESSING/FAILED jobs manually.

2.3.9 Reporting (RPT)

- FR-RPT-1 Daily, monthly and annual sales reports; profit report; inventory valuation report; low-stock report — optionally branch-scoped.
- FR-RPT-2 Tabular datasets (products, inventory, sales…) shall export to CSV, Excel and PDF.

2.3.10 Administration (ADMIN)

- FR-ADMIN-1 CRUD for users, branches, business profile and system settings.
- FR-ADMIN-2 Users may be assigned to multiple branches; branch switching updates the session context immediately.
- FR-ADMIN-3 Audit logs shall capture privileged actions with actor, action and timestamp; activity logs capture entity changes.

2.3.11 Synchronisation (SYNC)

- FR-SYNC-1 The desktop client shall pull branch-scoped snapshots (products, inventory, customers, etc.) into IndexedDB once a branch is selected.
- FR-SYNC-2 Mutations performed offline shall be pushed when connectivity returns; every sync run shall be logged with outcome.

2.4 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	Passwords bcrypt-hashed; JWT access+refresh with rotation; helmet, CORS allow-list, rate limiting, HPP and XSS-clean middleware enabled
NFR-2	Security	All privileged actions audit-logged; login history retained
NFR-3	Reliability	Message sends never lost: persistent queue with retries; SMTP failure falls back to structured console logging without blocking the request
NFR-4	Performance	List endpoints paginated (default 25–50 rows); search uses indexed ILIKE queries; POS search responds < 300 ms on LAN
NFR-5	Usability	Consistent glass-card UI; destructive actions confirmed; validation errors displayed inline/toast
NFR-6	Portability	Web client runs in evergreen browsers; desktop ships as Windows installer
NFR-7	Maintainability	Layered backend (routes → controllers → services → repositories); TypeScript front ends; oxlint clean
NFR-8	Data integrity	Money/quantity mutations execute inside DB transactions; PO totals recomputed server-side; soft deletes preserve history

2.5 External Interfaces

2.5.1 REST API

Base path /api/v1. JSON bodies; JWT bearer auth; standard envelope { success, data }. Representative resources:

Area	Endpoints (illustrative)
Auth	POST /auth/login, POST /auth/select-branch, POST /auth/refresh, POST /auth/forgot-password, POST /auth/reset-password
Catalogue	/products/search (POST), /categories, /suppliers, /uoms, /product-images
Inventory	/inventories/search (POST), /inventory-transactions, /inventory-transfers
Purchases	/purchases (+ item routes; guarded by DRAFT status)
Sales	/sales, carts
People	/users, /user-branches, /roles, /permissions, /customers, /customer-payments
Messaging	/notifications, POST /notifications (target USERS/ALL/BRANCH), /admin/queue, POST /admin/queue/:id/resend
Reports	/reports/daily-sales, /reports/monthly-sales, /reports/profit, /reports/inventory-report, /reports/low-stock
Sync	GET /sync/:entity/pull?branchId=…&since=…
Ops	/audit, /sessions, /system, expenses, price history, SMS balance

Interactive documentation: Swagger UI served by the backend (swagger-jsdoc + swagger-ui-express).

2.5.2 E-mail Interface

SMTP via Gmail (implicit TLS, port 465). Templates: password reset, account updated, role changed, notification fan-outs — wrapped in branded layout generated from business settings.

2.5.3 Hardware Interface (desktop)

HID barcode scanners via node-hid; keyboard-wedge scanners supported everywhere.

2.6 Acceptance Criteria (excerpt)

#	Scenario	Expected result
AC-1	Selecting branch B hides products stocked only in branch A	Products page shows exactly B's catalogue (verified: Main Shop 8, Second Store 2, Tema 4)
AC-2	Editing items of a received PO	HTTP 409 rejection
AC-3	Sending notification with target=BRANCH	One recipient row per active user of that branch only (verified 7 for ALL, 1 for single-user branch)
AC-4	Stock-out beyond availability	Rejected with clear message showing available stock
AC-5	SMTP outage during password reset	Request still succeeds; job queued and retried; admin can Resend

3. Project Chapter (Condensed Report)

Period covered: February – August 2026

3.1 Chapter 1 — Introduction

3.1.1 Background

Small and medium retail businesses that operate from more than one location commonly track stock with paper books or disconnected spreadsheets. This leads to stock-outs discovered only at the counter, unnoticed shrinkage, pricing inconsistencies between branches, and management decisions made on stale information. Cloud inventory tools exist, but many assume constant internet connectivity — an unrealistic assumption for shops in areas with unreliable power and network coverage.

3.1.2 Problem Statement

There is a need for an inventory system that:

- presents one shared, accurate view of stock per branch in real time;
- keeps selling even when the internet is down (offline POS);
- enforces accountability through roles, permissions and audit trails;
- reaches staff through multiple channels (in-app, e-mail, SMS) reliably;
- remains affordable and operable on ordinary Windows computers.

3.1.3 Objectives

- Design and implement a RESTful API backend that centralises all business data.
- Provide a web client for administrative work and a desktop client for the shop floor.
- Guarantee per-branch stock correctness with transactional updates.
- Support asynchronous notifications through a persistent retry-able queue.
- Document the system to handover quality.

3.1.4 Scope & Limitations

In scope: multi-branch inventory, POS (online + offline), purchasing, transfers, customers, expenses, notifications, reporting, RBAC administration.

Limitations: no double-entry accounting; SMS delivery depends on the gateway provider's balance; offline mode is limited to cached branch data on desktop.

3.2 Chapter 2 — Background & Related Work

Commercial systems (e.g., generic POS suites) typically bind merchants to subscription plans and online-only operation. Academic literature on retail information systems stresses three recurring requirements: data consistency across sites, resilience to connectivity loss, and traceability of stock movements. This

project applies those principles with an offline-first desktop design, a single authoritative database, and append-only movement ledgers. Where commercial tools charge per terminal, this system uses open technologies (Node.js, PostgreSQL, React, Electron), keeping running costs to hosting and an SMTP account.

3.3 Chapter 3 — System Analysis & Design

3.3.1 Methodology

An incremental, feature-driven process was used: each module (auth → catalogue → inventory → purchases → sales → notifications) moved through requirements → design → implementation → live-database verification before the next began, allowing early feedback from the business owner.

3.3.2 Architecture

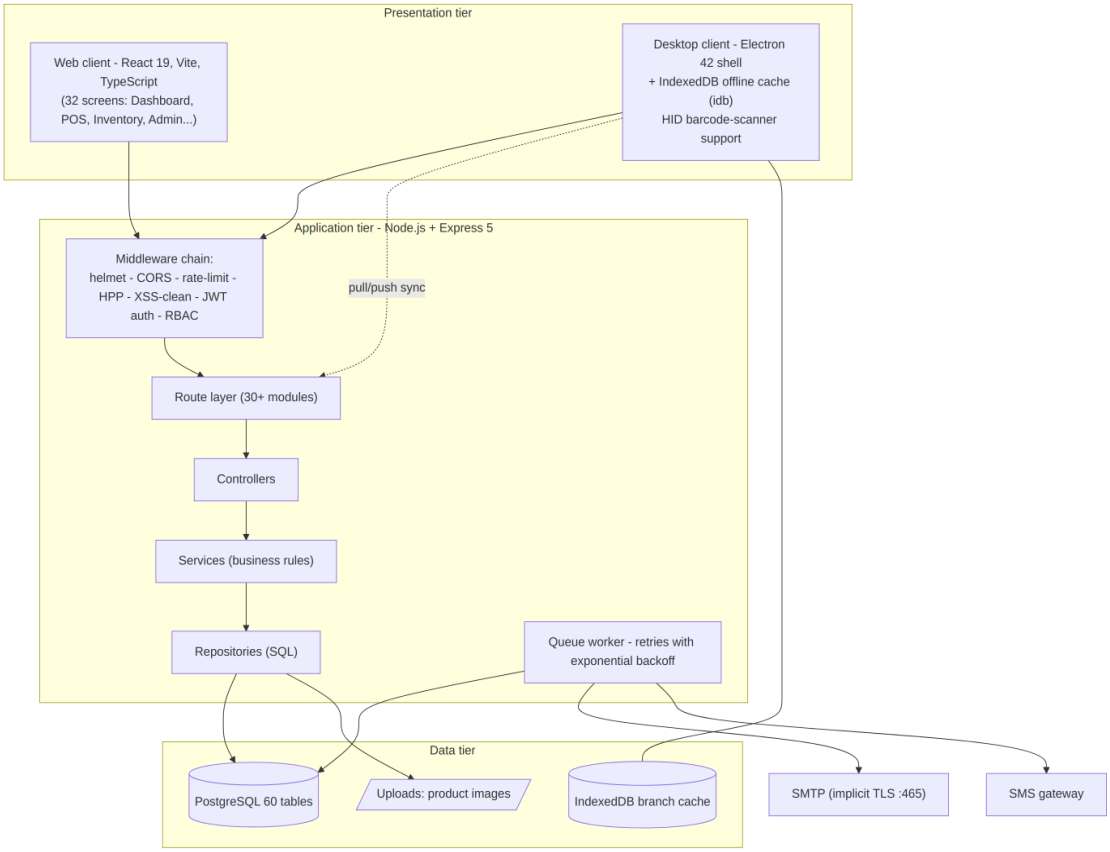

Figure 3.1 — Three-tier architecture: presentation, application, and data tiers

3.3.3 Database Design (core entities)

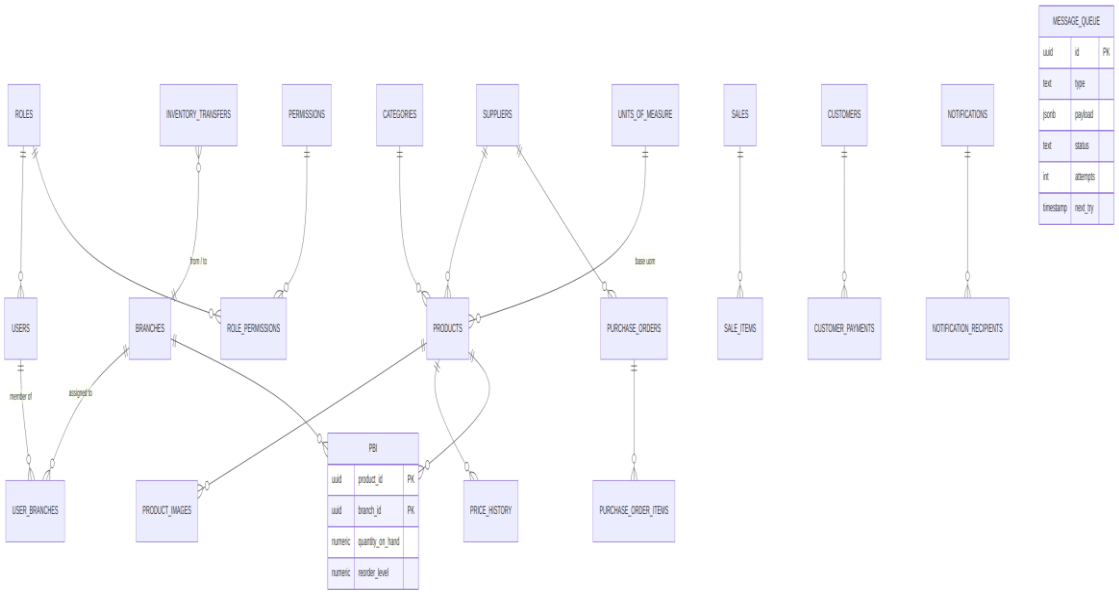

Figure 3.2 — Core entity-relationship diagram

Key decisions:

- Per-branch stock rows (product_branch_inventory) instead of a single quantity column make branch scoping a simple join and prevent cross-branch over-selling.
- Soft deletes (is_active) keep historical references valid; listings hide inactive records unless explicitly requested.
- Append-only ledgers for movements and price history provide full auditability.
- message_queue decouples message sending from request handling.

3.3.4 Security Design

Layered controls: bcrypt password hashing; short-lived access tokens plus rotating refresh tokens; tokens re-issued with branch context after selection; permission middleware per route group; hardened HTTP surface (helmet, rate limiting, parameter-pollution and XSS protection); privileged actions written to audit_logs; login history with IP/device details; one-hour single-use password-reset tokens delivered by e-mail.

3.3.5 Offline Strategy (desktop)

On branch selection the client pulls a branch-scoped snapshot (products, inventory, customers…) into IndexedDB. The POS reads from cache when offline and queues mutations; when connectivity returns it pushes them and logs the result in Sync Logs. The cache is invalidated and rebuilt whenever the selected branch changes.

3.4 Chapter 4 — Implementation & Testing

3.4.1 Technology Choices

Layer	Technology	Rationale
API	Node.js + Express 5	Ubiquitous, JSON-native, large ecosystem
Database	PostgreSQL	Transactional integrity, rich types (jsonb payloads)
Auth	jsonwebtoken + bcrypt	Proven stateless session pattern
Web UI	React 19 + Vite + TypeScript	Fast DX, type safety
Desktop	Electron 42 + idb + node-hid	Offline storage + scanner hardware access
Mail/SMS	Nodemailer (SMTP implicit TLS) + gateway	Reliable branded e-mail; SMS reach
Docs/QA	swagger-jsdoc/UI, Jest, Supertest, oxlint, tsc	Contract clarity and regression safety

3.4.2 Representative Implementations

- Branch-aware catalogue search: /products/search inner-joins product_branch_inventory when a branch filter is present; the controller normalises both camelCase and snake_case keys so every caller is served.
- PO lifecycle guard: item add/update/remove routes resolve the parent PO and reject changes once status leaves DRAFT (HTTP 409); totals are recalculated inside the same transaction as item mutations.
- Notification targeting: target ∈ {USERS, ALL, BRANCH} resolves to concrete recipient rows at send time (active users only), giving each recipient independent read-state; fan-out jobs flow through the queue.
- Queue reliability: failures increment attempts and schedule retries at attempt² × 30 s (cap 1 h); requeue resets status to PENDING; administrators can resend stuck PROCESSING/FAILED jobs from the Message Queue screen.
- Resilient e-mail: transport uses implicit TLS on port 465 (STARTTLS on 587 proved unreliable on target networks); on failure the service logs the full message and reports success:false so callers can enqueue a retry.

3.4.3 Testing Approach

- Endpoint verification against the live database — scripted controller invocations asserting exact row counts (e.g., branch-filtered catalogue: Main Shop 8, Second Store 2, Tema 4 products).
- Rule-based tests — DRAFT-only PO editing, availability checks on stock-out, notification audience resolution (ALL = 7 recipients vs BRANCH = 1), queue resend guards (409 on DONE, 404 on unknown id).
- Static analysis — oxlint clean on both clients; tsc -b build gate.

- Manual UAT — role-based walkthroughs with the business owner.

3.4.4 Defects Found & Fixed (highlights)

Defect	Root cause	Fix
Products identical across branches	snake/camelCase key mismatch silently dropped the branch filter	Server-side key normalisation + clients send branchId
Stock-out modal showed wrong availability	/inventories/search ignored snake_case filters	Controller normalisation + camelCase clients
PO total stayed 0.00 after item edits	Total never recomputed	Recalculate in-service + backfill script
E-mails never delivered ("Unexpected socket close")	STARTTLS on port 587 blocked mid-handshake	Implicit TLS 465 + configurable secure flag
Queue jobs stuck in PROCESSING forever	Retry path set next_try but not status back to PENDING	Requeue sets status='PENDING'

3.5 Chapter 5 — Conclusion & Future Work

3.5.1 Conclusion

The project delivers a complete, working multi-platform inventory system: a hardened RESTful API, a full-featured web client, and an offline-capable desktop POS, together with reliable asynchronous messaging and comprehensive documentation. All acceptance scenarios from the SRS were demonstrated against the live system.

3.5.2 Future Work

- Demand forecasting for automatic reorder suggestions.
- Native mobile client for stock counts.
- Multi-business tenancy for hosted deployments.
- Integration with mobile-money payment gateways at POS.
- Automated end-to-end browser tests in CI.

4. Prototype Description

The prototype is a fully working system, not a mock-up: every screen and flow described here exists in the codebase and runs against a live PostgreSQL database.

4.1 Prototype Overview

Aspect	Web client	Desktop client
Framework	React 19 + Vite 8 + TypeScript	Same UI stack inside Electron 42
Data access	REST (axios) over HTTPS	REST + IndexedDB offline cache (idb)
Offline support	None (requires connection)	Full POS operation offline, auto re-sync
Hardware	Keyboard-wedge barcode scanners	HID scanners via node-hid + wedge
Distribution	Served by Vite build / static host	Windows installer via electron-builder

Shared backend: Node.js + Express 5 REST API with PostgreSQL.

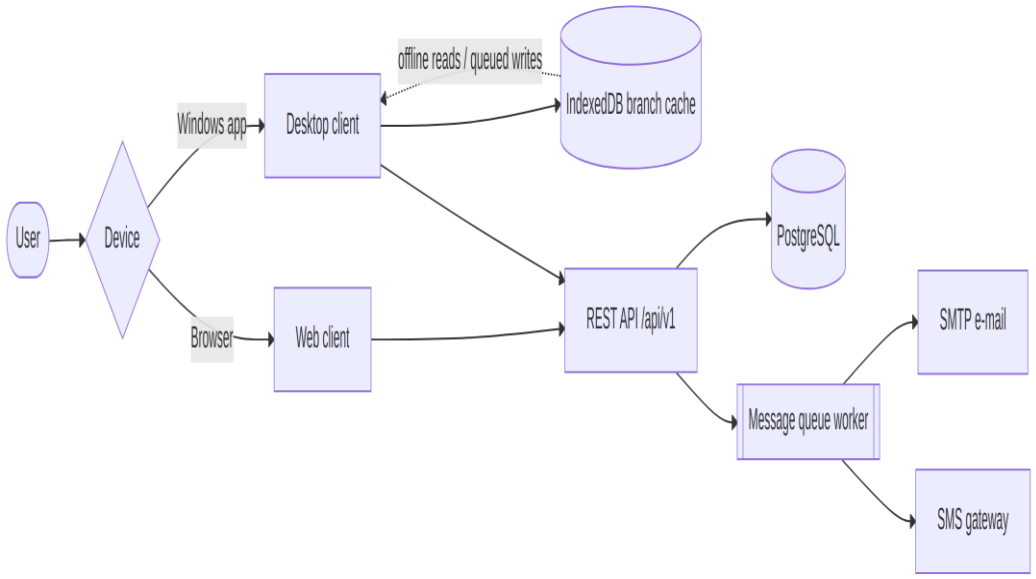

Figure 4.1 — Client/device routing to the shared REST API

4.2 Screen Inventory (web & desktop share the same set)

Operational: Dashboard · POS · Sales · Products · Categories · Inventory · Inventory History · Stock Movements · Transfers · Purchases · Suppliers · Customers · Price History · Low Stock Alerts · Expenses

Communication: Notifications · Create Notification · Message Queue · SMS Balance

Administration: Users · Roles · Branches · Business Settings · System Settings · Settings · My Profile · Audit Logs · Sync Logs · Sync

System: Login · Select Branch · Not Found (404)

4.3 Key Screen Walkthroughs

4.3.1 Login → Select Branch

- User signs in with username/e-mail + password.
- The Select a Branch screen lists assigned branches as cards (name, address, city; inactive ones disabled).
- Choosing a branch re-issues the auth token with branch context and opens the Dashboard. Single-branch users skip this screen automatically.
- The current branch is always visible in the top bar; switching returns to this screen and reloads branch-scoped data everywhere.

4.3.2 Dashboard

KPI cards and charts summarising sales and stock position for the selected branch, with quick links into daily operations.

4.3.3 Point of Sale (POS)

- Product search box accepts name/SKU/barcode (scanner-ready).
- Results show live availability for the current branch ("Available stock: N").
- Cart panel accumulates items with quantity steppers; totals update instantly.
- Checkout finalises the sale, decrements branch stock atomically and issues a receipt. On desktop this flow works with no internet: products come from cache and the sale syncs later.

4.3.4 Products

Paginated catalogue for the branch with search; create/edit forms cover SKU, barcode, category, supplier, base UoM and initial stock; CSV import maps columns to fields. Deactivated products are hidden unless "include inactive" is requested; exports honour the active filters.

4.3.5 Stock Movements

Ledger of IN/OUT/ADJUSTMENT/TRANSFER entries with actor and reference. "Record movement" modal searches the branch's products only, validates the quantity against available stock and blocks negatives — preventing overselling at entry time.

4.3.6 Purchases

Purchase orders list per branch. A PO in DRAFT allows free item editing; the moment it leaves DRAFT, item add/update/remove are rejected server-side (HTTP 409). Totals recalculate automatically on every change. Receiving increments branch stock; supplier payments record against the PO/supplier.

4.3.7 Transfers

Pick source/destination branches, add line items, confirm: source stock decrements, destination increments, both sides ledgered as one atomic action.

4.3.8 Notifications

- Create Notification: choose channels (in-app/e-mail/SMS) and audience — Specific users, All users, or Everyone in selected branch.
- Recipients resolve at send time to active users only; each gets independent read-state. Delivery flows through the message queue with retries.

4.3.9 Message Queue (Administrator)

Table of background jobs with status badges (PENDING/PROCESSING/DONE/FAILED), attempt counts and last error; filterable by status/type. Rows stuck in PROCESSING or FAILED show a Resend button that requeues the job instantly.

4.3.10 Reports

Daily/monthly/annual sales, profit, inventory valuation and low-stock reports; branch selector where relevant; export to CSV/Excel/PDF from the same screen.

4.3.11 Administration screens

Users (accounts + branch assignments), Roles (permission matrix), Branches, Business Settings (name/logo/address used across e-mail branding), System Settings, Audit Logs, Sync Logs, SMS Balance.

4.4 Core Flow Diagrams

4.4.1 POS Sale (online)

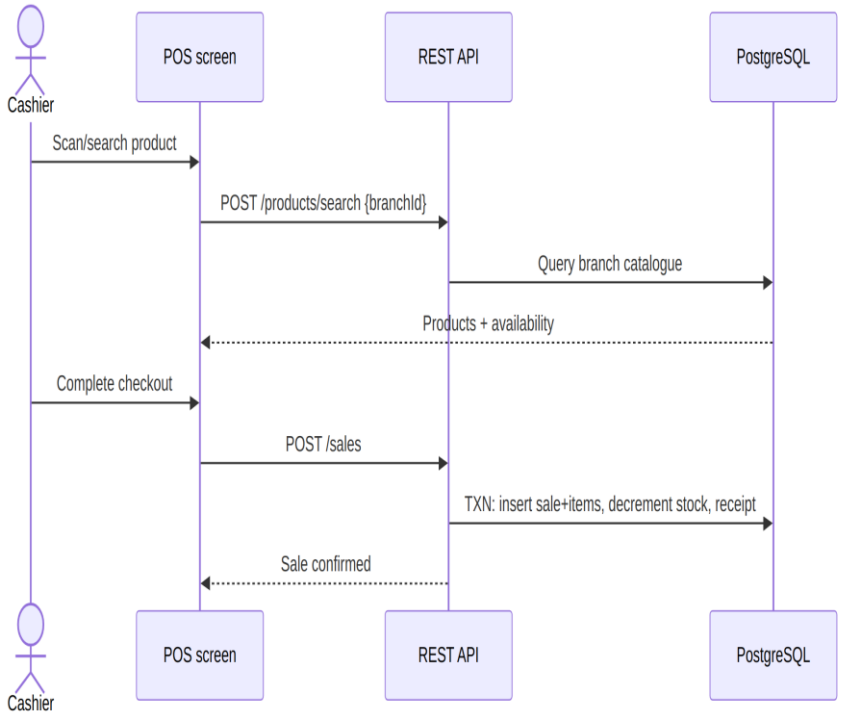

Figure 4.2 — POS checkout sequence

4.4.2 Purchase Order lifecycle

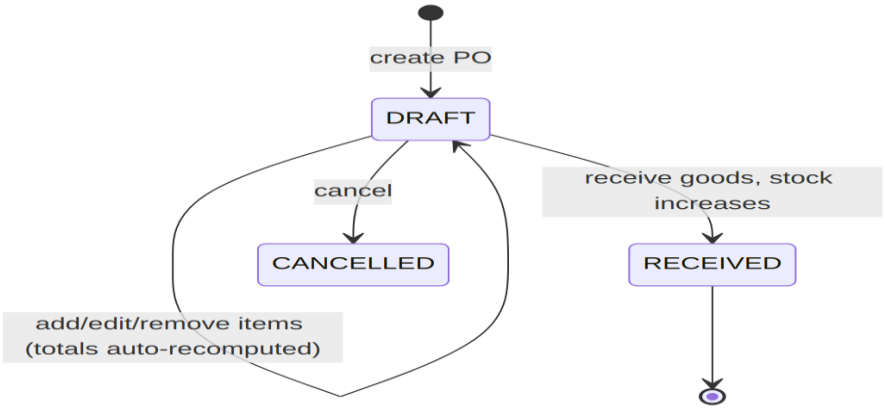

Figure 4.3 — Purchase order status lifecycle

Item edits are rejected once status ≠ DRAFT.

4.4.3 Password Reset

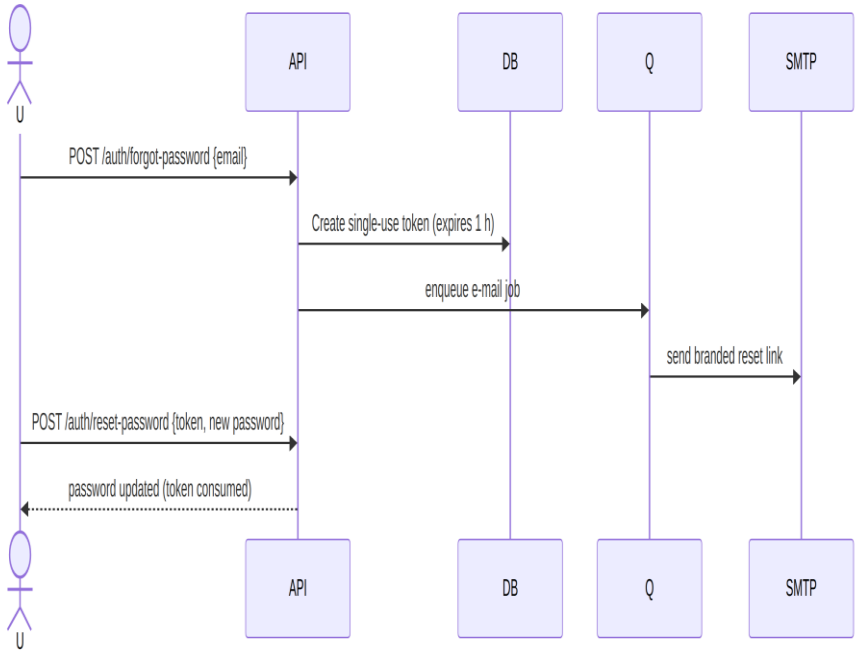

Figure 4.4 — Password reset sequence

If SMTP is unreachable, the job stays queued and retries with backoff; an administrator can also press Resend in the Message Queue screen.

4.4.4 Offline desktop sync

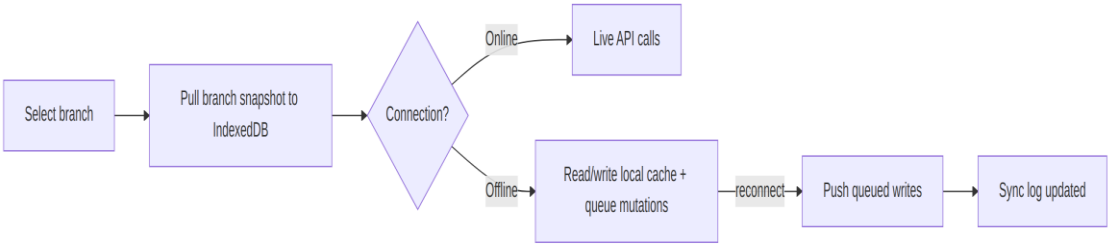

Figure 4.5 — Offline desktop synchronisation flow

4.5 What the Prototype Demonstrates

- Correct per-branch catalogues and stock (verified counts per branch).
- Safe concurrent stock changes via transactions and availability checks.
- Business-rule enforcement at the API layer (PO draft lock, resend guards).
- Reliable messaging: queue + retries + manual Resend + TLS-correct SMTP.
- True offline selling on the desktop client with transparent recovery.

5. Project Gantt Chart

Timeline: February – August 2026 (7 months; implementation phase June – August 2026 per repository history)

5.1 Gantt Chart

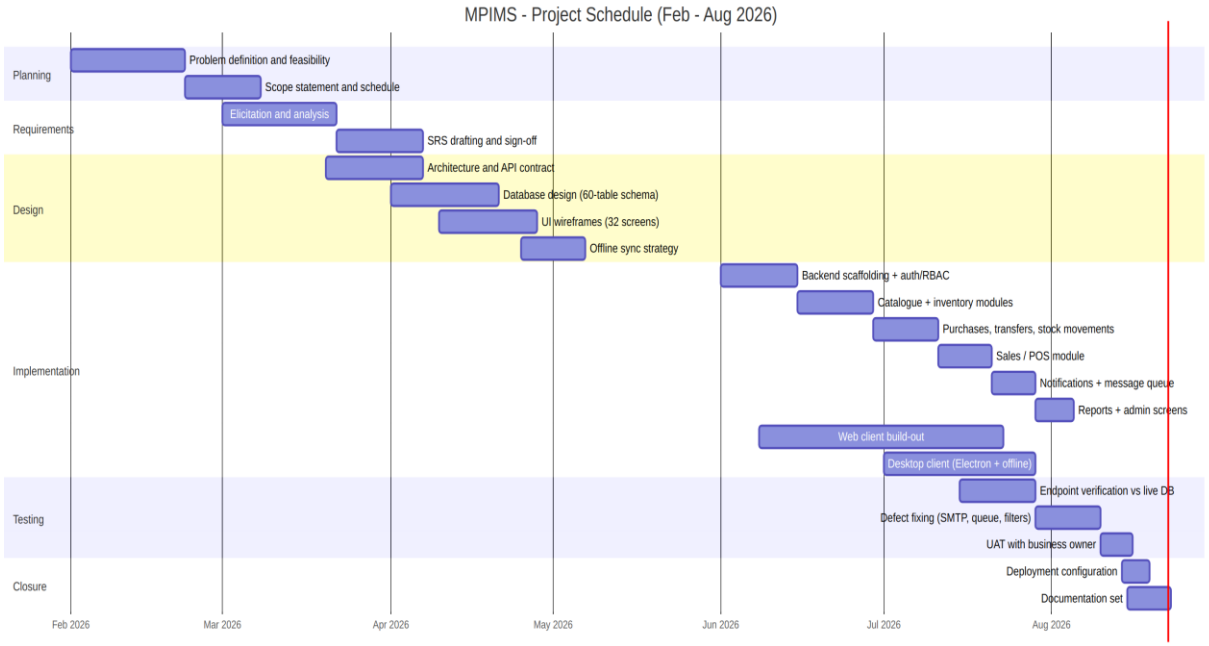

Figure 5.1 — MPIMS project schedule, February–August 2026

5.2 Schedule Table

Phase	Task	Start	End	Duration
Planning	Problem definition & feasibility	Feb 01	Feb 21	3 wks
Planning	Scope statement & schedule	Feb 22	Mar 07	2 wks
Requirements	Elicitation & analysis	Mar 01	Mar 21	3 wks
Requirements	SRS drafting & sign-off	Mar 22	Apr 06	2 wks
Design	Architecture & API contract	Mar 20	Apr 06	~2.5 wks
Design	Database design	Apr 01	Apr 20	~3 wks
Design	UI wireframes (32 screens)	Apr 10	Apr 27	~2.5 wks
Design	Offline sync strategy	Apr 25	May 06	~1.5 wks
Implementation	Backend scaffolding, auth, RBAC	Jun 01	Jun 14	2 wks
Implementation	Catalogue & inventory modules	Jun 15	Jun 28	2 wks

Phase	Task	Start	End	Duration
Implementation	Purchases, transfers, movements	Jun 29	Jul 10	~1.7 wks
Implementation	Sales / POS	Jul 11	Jul 20	~1.4 wks
Implementation	Notifications & message queue	Jul 21	Jul 28	1 wk
Implementation	Reports & admin screens	Jul 29	Aug 04	1 wk
Implementation	Web client build-out	Jun 08	Jul 22	~6.5 wks
Implementation	Desktop client (offline)	Jul 01	Jul 28	4 wks
Testing	Endpoint verification vs live DB	Jul 15	Jul 28	2 wks
Testing	Defect fixing	Jul 29	Aug 09	~1.7 wks
Testing	User acceptance testing	Aug 10	Aug 15	~1 wk
Closure	Deployment configuration	Aug 14	Aug 18	~0.7 wk
Closure	Documentation set	Aug 15	Aug 22	~1 wk

Backend, web-client and desktop tracks deliberately overlap; verification ran continuously against the live database rather than as a single late phase.

5.3 Milestones

ID	Milestone	Date	Exit criterion
M1	SRS approved	Apr 06	Signed requirements baseline
M2	Data model frozen	Apr 20	Schema applied to development DB
M3	Auth + inventory live on API	Jun 28	Branch-scoped endpoints verified
M4	Trading modules complete	Jul 20	POS sale round-trip works
M5	Messaging reliable	Jul 28	Queue retries verified; e-mail delivered
M6	Offline desktop proven	Jul 28	Airplane-mode sale syncs cleanly
M7	QA closed	Aug 09	Lint/type-check clean; defect list empty
M8	Handover pack issued	Aug 22	docs/ complete (WBS, SRS, chapter, prototype, gantt, manual)

5.4 Critical Path

Requirements → database design → backend auth/inventory → sales module → desktop offline layer → defect fixing → documentation. Slack exists in wireframing and reporting; none after M6 — closure activities were fixed to the August submission date.

6. User Manual

Web client & Desktop client — August 2026

6.1 Getting Started

6.1.1 Signing in

- Open the web address of your workplace system, or launch the desktop app.
- Enter your username or e-mail and password → Sign in.
- Select your branch — click the card for the branch you are working from.
    - If you belong to only one branch, it is selected automatically.
    - The chosen branch appears in the top bar; everything you see is scoped to it.

Forgot your password? Click Forgot password, enter your e-mail and follow the link we send you (valid for 1 hour). If no e-mail arrives within a few minutes, ask an Administrator to check the Message Queue page and press Resend on your reset job.

6.1.2 Switching branch

Click the branch name in the top bar → pick another branch on the selection screen. All lists reload for the new branch automatically — products stocked only elsewhere will not appear.

6.1.3 Understanding roles

Role	Can typically do
Business Owner	Everything: settings, all branches, reports
Administrator	Users, roles, branches, message queue, audit logs
Branch Manager	Branch inventory, purchases, transfers, approvals
Storekeeper	Receiving goods, stock counts/movements
Cashier	POS sales only

If a menu is missing for you, your role does not include that permission — see your Administrator.

6.2 Daily Selling (POS)

- Go to POS.
- Scan a barcode or type a product name/SKU in the search box. Results show live stock for your branch ("Available stock: 24").
- Click a product (or press its row) to add it to the cart; adjust quantities.
- Press Checkout / Complete sale. Stock is reduced instantly and a receipt is created.

Desktop offline mode: if the internet drops, keep selling as normal using the cached catalogue — sales queue locally and sync automatically once you are back online. Check Sync Logs (Administration) to confirm queued sales went through.

6.3 Managing Products

- Products page lists your branch's catalogue with search and pagination.
- Add product: New Product button → fill SKU, barcode, name, category, supplier, base unit, opening stock → save.
- Edit / deactivate: open a product; deactivation hides it everywhere without deleting history. Deactivated items can be re-enabled by an administrator (they stay hidden until then).
- Import: use Import CSV on the Products page (columns: Name, SKU, Category, Supplier, Base UoM…).
- Export: the Export menu downloads the current list to CSV/Excel/PDF.

6.4 Stock Operations

6.4.1 Record a stock movement

- Open Stock Movements → Record Movement.
- Choose IN (goods in) or OUT (removals/damages).
- Search the product — the picker shows only your branch's products and their available quantity.
- Enter quantity and reason/note → save. The system rejects quantities larger than available stock.

6.4.2 Transfer stock between branches

- Transfers → New Transfer.
- Pick source and destination branches, add products and quantities.
- Confirm. Source stock decreases and destination stock increases as one safe operation; both ledgers record the movement.

6.4.3 Watch low stock

Low Stock Alerts lists every item at or below its reorder level so you can reorder before running out.

6.5 Buying Stock (Purchases)

- Purchases → New Purchase Order (select your branch).
- Add line items while the PO is in DRAFT — totals update automatically.
- Once finalised, items are locked (edits return "only DRAFT purchase orders can be modified").
- When goods physically arrive, mark the PO received — branch stock increases.
- Record supplier payments against the PO/supplier as they are made.

6.6 Customers & Payments

- Customers: store business name, contact person, phone, e-mail. At least one identifier is required.
- Record customer payments and track balances on the customer's profile.

6.7 Expenses

Record daily expenses under categories (Expenses) for branch-level cost visibility alongside sales reports.

6.8 Notifications

6.8.1 Reading notifications

Click the bell icon to see your notifications; mark them read or delete them.

6.8.2 Sending notifications (permission required)

- Create Notification.
- Fill title and message; choose type and channels (In-app / E-mail / SMS).
- Choose the audience:
    - Specific users — pick people from the list;
    - All users — everyone active in the business;
    - Everyone in selected branch — active staff of one branch.
- Send. Delivery happens in the background with automatic retries; recipients each get their own copy.

6.9 Administration

6.9.1 Users & Roles

- Create accounts, assign a role and branch memberships (Users).
- Shape what each role may do via the permission matrix (Roles).

6.9.2 Branches & business profile

Manage branch records (Branches) and the company name/logo/contact details used across the system and e-mails (Business Settings).

6.9.3 Message Queue (Administrators)

Shows background e-mail/SMS jobs: status, attempts, last error.

- Filter by status/type to investigate problems.
- Press Resend on any PROCESSING/FAILED job to requeue it immediately.
- DONE jobs cannot be resent (by design).

6.9.4 Audit Logs

Review who did what and when — privileged actions, logins and entity changes.

6.10 Reports

Open Reports for:

- Daily / Monthly / Annual sales,
- Profit report,

- Inventory valuation,
- Low-stock report.

Use the branch selector where available, then export with the CSV/Excel/PDF buttons.

6.11 Troubleshooting

Problem	What it means	What to do
"No branch assigned" after login	Account has no branch membership	Contact an Administrator to add you under Users → Branches
Product missing from POS/list	It belongs to another branch, or was deactivated	Switch branch; or ask admin to re-enable it
"Only DRAFT purchase orders can be modified"	PO already finalised/received	Create a new PO for corrections
Stock-out rejected although shelf looks full	System tracks recorded stock per branch	Record an IN movement or stock count first
Reset e-mail never arrives	SMTP hiccup at send time	Admin: Message Queue → find the job → Resend
Desktop sale "pending sync"	Working offline	Keep selling; verify later in Sync Logs
Wrong branch data showing	Stale selection	Switch branch away and back; lists reload scoped to the branch

6.12 Tips

- Barcodes: any USB scanner that "types" text works on both clients; the desktop app additionally supports HID-mode scanners.
- Use search + filters before exporting so files contain exactly what you need.
- Deactivate instead of delete — reports and history stay correct.